# Entity interpolation for high speed projectiles is wildly inaccurate

**MC-299627** `Projectiles`

| | |
|---|---|
| **Status** | Resolved / Fixed |
| **Affected** | 1.21.7 |
| **Fixed in** | 25w32a |
| **Reported** | 2025-07-10 |

## STEPS TO REPRODUCE

1. Launch Minecraft and create a Creative world with default settings.
2. Execute `/summon minecraft:arrow ~ ~1 ~ {Motion:[0.0,1.0,0.0]}`.
3. Execute `/summon minecraft:arrow ~ ~1 ~ {Motion:[0.1,1.0,0.0]}`.
4. Execute `/summon minecraft:arrow ~ ~1 ~ {Motion:[0.0,10.0,0.0]}`.
5. Execute `/summon minecraft:arrow ~ ~1 ~ {Motion:[1.0,10.0,0.0]}`.
6. Observe the arrows as they move through the world.
7. Compare the arrows moving at approximately 1 block per tick with the arrows moving at the higher speeds.
8. Observe that the rendered positions of the high-speed arrows become substantially inaccurate compared with their actual entity positions, while the discrepancy is much less noticeable for the lower-speed arrows.

## OBSERVED BEHAVIOUR

### Actual behavior

At high projectile velocities, the client-rendered position of an arrow does not accurately follow the arrow entity's actual position. The discrepancy is minor at approximately 1 block per tick, but becomes increasingly pronounced as the projectile's velocity increases. For example, arrows moving at approximately 10 blocks per tick visibly diverge from their actual hitboxes during flight.

The issue is most apparent when comparing the server/entity position with the client-rendered position: the entity's actual hitbox advances along its expected trajectory, while the rendered arrow is interpolated using an inaccurate motion value. This behavior has been observed with arrow entities and does not appear to affect entities such as TNT in the same way.

The discrepancy is caused by the velocity values sent in `ClientboundSetEntityMotionPacket`. Each motion component is clamped to the range `[-3.9, 3.9]` before being multiplied by `8000` and written to the packet as a signed short:

```java
double x = Mth.clamp(vec3.x, -3.9, 3.9);
double y = Mth.clamp(vec3.y, -3.9, 3.9);
double z = Mth.clamp(vec3.z, -3.9, 3.9);
```

Consequently, a projectile with a motion component of `10.0` is transmitted to the client as `3.9`. For example:

- `[0.0, 1.0, 0.0]` is transmitted accurately.
- `[0.1, 1.0, 0.0]` is transmitted accurately.
- `[0.0, 10.0, 0.0]` is transmitted as approximately `[0.0, 3.9, 0.0]`.
- `[1.0, 10.0, 0.0]` is transmitted as approximately `[1.0, 3.9, 0.0]`.

The client therefore receives and uses a substantially lower velocity than the projectile actually has, resulting in visibly incorrect interpolation and rendering for high-speed projectiles.

## EXPECTED BEHAVIOUR

High-speed projectiles should be rendered at positions that closely match their actual entity positions on the server each frame. Arrows traveling at any tested velocity should follow their true trajectory smoothly, without the client-rendered hitbox lagging behind, deviating from, or stopping short of the actual hitbox. The accuracy should

remain consistent as projectile speed increases.

### ENVIRONMENT

Minecraft Java Edition Version 1.21.7

### ORIGINAL DESCRIPTION

Whilst testing arrows launched at high speeds I've noticed that the motion of the arrows seems really strange. After investigating further I have found there are wild inaccuracies in the way projectiles are rendered to the client each frame. This appears to only affect projectile entities and not other types of entities like TnT.

I have included a video clearly demonstrating the issue. In the video the green hitbox represents the actual position of the projectile entity, whilst the white hitbox follows the entity as it is rendered for the client. For very low speeds of around 1 block per tick its not very noticable, but once you start getting to higher speeds the discrepancy becomes very pronounced.

To reproduce the setup you can use the following commands:

summon minecraft:arrow ~ ~1 ~

{Motion:[0.0,1.0,0.0]}

summon minecraft:arrow ~ ~1 ~

{Motion:[0.1,1.0,0.0]}

summon minecraft:arrow ~ ~1 ~

{Motion:[0.0,10.0,0.0]}

summon minecraft:arrow ~ ~1 ~

{Motion:[1.0,10.0,0.0]}

Thanks to Velizar we now know exactly what part of the code is causing this issue. When the game goes to compile entity data to transmitto the client. It clamps the motion values to a range of [-3.9,3.9] and multiplies by 8000 to ensure the data is transmitted in 32 signed integer format. This seems like a completely unnecessary networking optimization that just detracts from the visual accuracy of gameplay features. Included is the code that causes this issue.

```
package net.minecraft.network.protocol.game;
import net.minecraft.network.FriendlyByteBuf;
import net.minecraft.network.codec.StreamCodec;
import net.minecraft.network.protocol.Packet;
import net.minecraft.network.protocol.PacketType;
import net.minecraft.util.Mth;
import net.minecraft.world.entity.Entity;
import net.minecraft.world.phys.Vec3;
public class ClientboundSetEntityMotionPacket implements Packet<ClientGamePacketListener> {
public static final StreamCodec<FriendlyByteBuf, ClientboundSetEntityMotionPacket> STREAM_CODEC = Packet.codec(
ClientboundSetEntityMotionPacket::write, ClientboundSetEntityMotionPacket::new
);
private final int id;
private final int xa;
private final int ya;
private final int za;
publ
```

## MANUAL RESULT

| | |
|---|---|
| **Version used** | |
| **Reproduced?** | YES / NO / PARTIAL |
| **Crash file** | |
| **Notes** | |
| | |
| **Tested by / date** | |

Live issue: https://bugs.mojang.com/browse/MC/issues/MC-299627